Concurrencia y Paralelismo

Grado en Informática 2026

Práctica 2 – Librerías mutex / semáforos

Ejercicio 1 (**Mutex recursivos**) Un mutex recursivo permite al thread que ha bloqueado un mutex bloquearlo un número arbitrario de veces. El mutex solo queda desbloqueado cuando se ha hecho el mismo número de unlocks que de locks.

Es posible implementar mutex recursivos utilizando como base los de la librería pthread, más la función `pthread_t pthread_self()`, que devuelve el identificador del thread que la llama. Implemente estos mútex en `rec_mutex.c`, siguiendo la interfaz definida en `rec_mutex.h`.

Compruebe que funcionan correctamente escribiendo un programa de prueba en el que varios threads trabajan con un array de un cierto tamaño inicializado con los números de 0 en adelante. Cada thread itera generando dos posiciones aleatorias e intercambiandolas. Cada posición debe estar protegida por su propio rec_mutex.

Ejercicio 2 (**Mutex prioridades**) Implemente, utilizando como base los mutex de la librería pthread, una librería donde se puedan bloquear mutex con distinta prioridad. El número de prioridades distintas se configura al crear el mutex, y puede asumirse que va a ser un número razonablemente pequeño.

La operación `mutex_prio_lock(mutex_t *m, int prio)` intenta bloquear el mutex con prioridad `prio`. El mutex solo podrá ser bloqueado si está libre, o si no hay ningún otro thread intentando hacer un bloqueo con mayor prioridad.

Diseñe e implemente un programa de prueba que compruebe el funcionamiento correcto de estos mutex.

Ejercicio 3 (**Mutex Read/Write**) Una forma de resolver el problema de lectores/escritores es implementar un mutex donde se puedan hacer bloqueos para lectura (`mutex_read_lock`) y para escritura (`mutex_write_lock`). Un bloqueo de escritura obtiene acceso exclusivo a la zona compartida, pero un bloqueo de lectura es compatible con otros bloqueos de lectura.

Implemente una librería de mutex con bloqueos de lectura/escritura según la intefaz definida en `rw_mutex.h`.

Ejercicio 4 (**Semáforos**) Vamos a implementar, utilizando los mutex y condiciones de la librería pthread, nuestra propia librería de semáforos. Para ello, implemente la interfaz definida en el fichero `sem.h` dentro del fichero `sem.c`. En la estructura `sem_t` solo puede usarse tipos básicos de C, mutex y condiciones.

Implemente el problema del barbero usando semáforos descrito en clase utilizando la librería que acabamos de definir en un fichero `barber.c`. Tanto los barberos como los clientes deberían imprimir un mensaje cuando vayan a cortar el pelo indicando su número de barbero / cliente.

Entrega

La fecha límite de entrega es el 6 de marzo. Para crear el repositorio y acceder al código inicial, acceda a github classroom en https://classroom.github.com/a/wA902tAy. El nombre de los grupos de prácticas debe ser el login de ambas personas, separados por un guión.